

VIHAND GRADE

ĐẶC TẢ KỸ THUẬT HỆ THỐNG

Hệ thống nhận dạng, sửa lỗi và chấm chính tả chữ viết tay tiếng Việt cho học sinh tiểu học

Thuộc tính	Giá trị
Phiên bản tài liệu	1.0 - khảo sát mã nguồn hiện có
Phạm vi	Toàn bộ workspace Web_sua_loi
Kiến trúc	Next.js + Prisma/SQLite + Gemini + FastAPI ViT5 + MCP Xiaozhi
Ngày lập	25/07/2026
Trạng thái	Tài liệu hiện trạng và khuyến nghị kỹ thuật

Tài liệu mô tả thiết kế đang được triển khai; không khẳng định rằng các kiểm soát bảo mật hoặc triển khai vận hành đã được hoàn tất.

1. Mục đích, phạm vi và thuật ngữ

Tài liệu này đặc tả cấu trúc kỹ thuật hiện hữu, luồng dữ liệu, hợp đồng API, mô hình dữ liệu, các dịch vụ AI và quy trình triển khai của ViHand Grade.

1.1 Mục tiêu nghiệp vụ

- Cho phép giáo viên tải ảnh bài viết tay hoặc nhập văn bản để nhận dạng, sửa lỗi chính tả và chấm điểm.
- Lưu bài chấm theo học sinh, lớp và bài tập để tra cứu lịch sử, báo cáo và quản trị.
- Hỗ trợ đọc chính tả qua trợ lý Xiaozhi, lưu phiên đọc và nhật ký hội thoại.
- Tối ưu chạy cục bộ/Raspberry Pi bằng microservice ViT5 trên CPU, nhưng vẫn có Gemini làm OCR và cơ chế dự phòng.

1.2 Phạm vi mã nguồn

Khu vực	Vai trò
app/	Next.js App Router: giao diện, layout, route handlers API, service worker.
components/, hooks/	Thành phần giao diện theo shadcn/Radix và hook responsive/toast.
lib/	Prisma client, xử lý ảnh Jimp, tiện ích và kiểu dùng chung.
prisma/	Schema SQLite, migrations và dữ liệu seed.
python_service/	FastAPI ViT5, sửa lỗi và chấm điểm SequenceMatcher/Levenshtein.
mcp_service/	FastAPI JSON-RPC/MCP, kết nối Xiaozhi và gọi API Next.js.
02_Kịch_ban_Thuc_nghiem/	Dataset ảnh, benchmark, notebook và kịch bản kiểm thử nghiên cứu.
01_Bao_cao_Nghien_cuu/, 03_Scripts_Trien_khai/	Tài liệu báo cáo và script triển khai Raspberry Pi/máy chủ.

1.3 Thuật ngữ

Thuật ngữ	Diễn giải
OCR	Nhận dạng văn bản từ ảnh. Hệ thống dùng Gemini Vision.
ViT5	Mô hình seq2seq tiếng Việt dùng để đề xuất văn bản đã sửa.
MCP	Model Context Protocol; cầu nối để Xiaozhi gọi công cụ lưu/lấy phiên đọc.
Barem	Thang điểm 10: chính tả 4, hình thức 3, nội dung 2, sáng tạo 1.
Fallback	Cơ chế chuyển sang Gemini chấm văn bản khi dịch vụ ViT5 không sẵn sàng.

2. Kiến trúc tổng thể

Hệ thống được thiết kế theo mô hình web monolith Next.js kết hợp hai microservice Python: ViT5 cho suy luận cục bộ và MCP cho trợ lý đọc chính tả.

2.1 Sơ đồ thành phần logic

Tầng	Thành phần	Trách nhiệm	Giao tiếp
Trình duyệt	Trang đăng nhập, giáo viên, học sinh, quản trị	Tải ảnh, xem kết quả, quản lý dữ liệu, điều khiển camera.	HTTPS/JSON tới Next.js
Ứng dụng	Next.js 16 Route Handlers	Xác thực hiện trạng, CRUD, OCR, tiền xử lý, điều phối chấm và PWA.	Prisma, HTTP nội bộ
AI đám mây	Google Gemini 3.1 Flash Lite	OCR ảnh chữ viết tay; chấm văn bản dự phòng.	Google GenAI SDK
AI cục bộ	FastAPI ViT5	Sửa văn bản, phân loại lỗi và tính điểm tắt định.	POST /grade, /preload
Dữ liệu	SQLite qua Prisma	User, Class, Grade, DictationSession, DictationLog.	File DB theo DATABASE_URL
Trợ lý	MCP FastAPI + Xiaozhi	Nhận/gửi JSON-RPC, lưu hoặc tra cứu phiên đọc.	WebSocket và HTTP

2.2 Luồng chấm bài

- 1. Giáo viên chọn ảnh; client đọc Data URL và tự gọi /api/preprocess.
- 2. API tiền xử lý dùng Jimp để chuẩn hóa ảnh và trả ảnh JPEG base64 kèm báo cáo chất lượng.
- 3. Client nén ảnh JPEG tối đa 1.280 px trước khi POST /api/ocr.
- 4. Gemini trả original_text (bản nhận dạng) và gemini_fixed_text (tham khảo).
- 5. Client gửi original_text đến /api/grade; route handler gọi FastAPI ViT5 với timeout 120 giây, retry một lần sau 10 giây.
- 6. ViT5 sửa văn bản, so khớp từ, phân loại lỗi, trả breakdown/điểm/nhận xét; nếu lỗi, Next.js dùng Gemini fallback.
- 7. Giáo viên lưu kết quả bằng POST /api/grades vào SQLite.

2.3 Ràng buộc vận hành

Mã frontend gọi API cùng origin. ViT5 và MCP mặc định chạy localhost:8000 và localhost:8200; cổng web là 3000. Docker hiện đóng gói Next.js độc lập, không đóng gói Python service hoặc MCP trong cùng image.

3. Thành phần giao diện web

Ứng dụng dùng Next.js App Router, React 19 và Tailwind CSS v4. Các control cơ bản nằm trong components/ui theo phong cách shadcn/Radix.

3.1 Layout và trang

Đường dẫn	Tệp	Chức năng
/	app/page.tsx	Đăng nhập; lưu hồ sơ trả về vào localStorage rồi chuyển theo role.
/register, /admin-register	app/register/page.tsx; app/admin-register/page.tsx	Đăng ký người dùng và quản trị.
/teacher	app/teacher/page.tsx	Bảng điều khiển giáo viên.
/teacher/grade	app/teacher/grade/page.tsx	Luồng tải/camera, tiền xử lý, OCR, chấm và lưu bài.
/teacher/dictation	app/teacher/dictation/page.tsx	Quản lý/tra cứu phiên đọc chính tả.
/teacher/reports	app/teacher/reports/page.tsx	Báo cáo theo bài chấm.
/student, /student/history	app/student/*	Không gian học sinh và lịch sử.
/admin/*	app/admin/*	Quản trị dashboard, users, classes, statistics, settings, system.

3.2 Root layout và PWA

app/layout.tsx khai báo ngôn ngữ vi, font Roboto subset Vietnamese, metadata/icon, theme-color và Vercel Analytics ở production. Layout đăng ký service worker qua script inline; handler app/sw.js/route.ts cung cấp tệp worker. Cần áp CSP nghiêm ngặt nếu giữ script inline.

3.3 Trạng thái và tích hợp client

Trang chấm bài là client component lớn. Nó quản lý ảnh gốc/ảnh xử lý, văn bản OCR, trạng thái xử lý, cấu hình trừ điểm, thông tin học sinh/lớp, camera stream và kết quả chấm bằng useState. Khi mount, trang lấy danh sách lớp, warm-up ViT5 và lọc học sinh theo lớp. Các lời gọi fetch không mang access token; điều này là một hạn chế bảo mật quan trọng.

3.4 Thư viện phụ thuộc nổi bật

Nhóm	Thư viện	Dùng cho
Framework	next 16.2.4, react 19, typescript	SSR/App Router và giao diện.
UI	Radix UI, lucide-react, sonner, recharts	Control truy cập được, icon, thông báo, biểu đồ.
Dữ liệu	Prisma 5, better-sqlite3, libsql client	ORM và adapter SQLite/libSQL.
AI/ảnh	@google/genai, Jimp	Gemini và xử lý ảnh server-side.
Style	Tailwind CSS 4, tw-animate-css	CSS utility và animation.

4. Đặc tả API Next.js

Các API nằm trong `app/api/*/route.ts`. Request/response đều dùng JSON, ngoại trừ base64 được truyền như chuỗi trong JSON. Hiện tại chưa có middleware xác thực hoặc phân quyền server-side.

4.1 Xác thực và người dùng

Phương thức / endpoint	Input chính	Xử lý / output
POST /api/auth/login	username, password	Tìm User, so sánh password, trả user và danh sách lớp giáo viên.
POST /api/auth/register	name, username, password, className	Tạo học sinh/giáo viên theo dữ liệu request.
POST /api/auth/admin-register	name, username, password, confirmPassword, secretKey	Kiểm secretKey rồi tạo admin.
GET /api/users?role=...	role tùy chọn	Liệt kê User.
POST /api/users	Thông tin User	Tạo User.
PATCH /api/users/:id	body bất kỳ	Cập nhật trực tiếp User theo body.
DELETE /api/users/:id	id URL	Xóa User.

4.2 Lớp, bài chấm và đọc chính tả

Phương thức / endpoint	Trách nhiệm
GET /api/classes	Trả lớp kèm số học sinh, giáo viên và aggregate điểm.
POST /api/classes	Tạo lớp; kiểm tra name duy nhất.
PATCH, DELETE /api/classes/:id	Cập nhật hoặc xóa lớp.
GET /api/grades	Lấy danh sách kết quả chấm, hỗ trợ lọc theo query trong implementation.
POST /api/grades	Lưu snapshot bài chấm gồm văn bản, corrections, breakdown, điểm và ảnh base64.
GET, DELETE /api/grades/:id	Xem hoặc xóa một Grade.
GET, POST /api/dictation/sessions	Liệt kê hoặc tạo phiên đọc, có logs.
GET /api/dictation/sessions/:id và /logs	Chi tiết phiên và nhật ký hội thoại.

4.3 Xử lý AI

Endpoint	Input	Output và xử lý
POST /api/preprocess	imageBase64	processedImageBase64 JPEG và quality {is_good, warnings, blur_score, brightness, resolution, ...}.
POST /api/ocr	imageBase64, mimeType	text là original_text OCR; gemini_fixed_text chỉ để tham khảo; tokenCount và processingTimeMs.
POST /api/grade	studentText, penalty_per_error, hình_thuc, noi_dung	Kết quả ViT5 hoặc Gemini: original/fixed text, corrections, breakdown, score, rating, feedback, engine.
GET /api/vit5-warmup	-	Gọi service Python preload để giảm thời gian lần chấm đầu.

4.4 Quy ước lỗi

Route handlers phần lớn trả { error: string } cùng HTTP 400 cho input thiếu, 401/403 cho đăng nhập, 409 khi trùng username/lớp và 500/503 khi lỗi hạ tầng AI. Chuẩn hóa error envelope, mã lỗi máy đọc được và correlation ID là bước nên thực hiện tiếp theo.

5. Mô hình dữ liệu và lưu trữ

Prisma datasource sử dụng SQLite với DATABASE_URL. Schema chỉ định ID cuid dạng chuỗi, timestamp tạo tự động và không mô hình hóa đầy đủ foreign key cho User/Class/Grade.

5.1 Bảng User

Trường	Kiểu / mặc định	Ý nghĩa
id	String cuid, PK	Định danh người dùng.
name, username	String; username unique	Thông tin hiển thị và đăng nhập.
password	String, default 123456	Mật khẩu hiện đang lưu rõ; không an toàn.
role	String, default student	teacher student admin theo quy ước code.
className, active, createdAt	String, Boolean, DateTime	Lớp text, trạng thái và thời điểm tạo.

5.2 Bảng Class và Grade

Model	Trường chính	Nhận xét
Class	id, name unique, grade, teacherId, createdAt	teacherId là String không có relation Prisma; xóa User có thể tạo tham chiếu mồ côi.
Grade	studentName, assignmentTitle, className, originalText, fixedText, corrections, score, scoreNum, scoreBreakdown, feedback, rating, imageBase64	Lưu snapshot đầy đủ để tra cứu. corrections/breakdown là String JSON, ảnh base64 có thể làm phình SQLite.
DictationSession	title, passage, className, teacherName, status, summary	Một phiên đọc chính tả, relation 1-n với DictationLog.
DictationLog	sessionId FK, speaker, content, createdAt	Nhật ký theo speaker xiaozhi/teacher/student; onDelete Cascade.

5.3 Khuyến nghị mô hình hóa

- Đổi role thành Prisma enum; liên kết Class.teacherId -> User.id và Grade.studentId/Class.id thay cho tên dạng String.
- Tách blob ảnh sang object storage hoặc filesystem có URL/hash; giới hạn kích thước request/ảnh.
- Dùng Json column nếu chuyển sang PostgreSQL hoặc xác định DTO validation cho chuỗi JSON hiện hữu.
- Tạo migration cho chỉ mục lọc theo className, createdAt, student và cơ chế backup/retention cho dữ liệu trẻ em.

6. Dịch vụ AI và thuật toán chấm

ViT5 service là FastAPI, tải model chamdentimem/ViT5_Vietnamese_Correction theo lazy-load và cache trong bộ nhớ. Dịch vụ được tối ưu để chạy CPU/Raspberry Pi.

6.1 Hợp đồng FastAPI ViT5

Endpoint	Mô tả
GET /health	Trạng thái service, model loaded/quantized và thông tin phục vụ health-check.
POST /preload	Ép tải model trước khi người dùng chấm.
POST /grade	Nhận text, penalty_per_error, hình_thuc, noi_dung; chạy ViT5 và grade_with_levenshtein.

6.2 Sửa lỗi ViT5

- Tiền xử lý teencode và dấu câu; loại từ lặp liền kề nhưng giữ danh sách từ láy cố ý.
- Phân biệt thơ/văn xuôi để giữ xuống dòng; nhận diện tiêu đề/nhãn để không đưa vào model.
- Chia văn xuôi thành chunk tối đa 160 ký tự; batch 4, max token động tối đa 256.
- Dùng dynamic INT8 quantization cho Linear layer, PyTorch inference mode và ThreadPoolExecutor 1 worker.
- Phòng hallucination: fallback nếu lặp cụm, output >1.5 lần input hoặc <0.5 lần input; cache tối đa 200 kết quả không fallback.

6.3 Chấm và phân loại lỗi

grade_with_levenshtein chuẩn hóa dấu câu, tách từ rồi dùng difflib.SequenceMatcher giữa văn bản đã sửa và bản học sinh. Opcode replace/insert/delete được biến thành corrections. classify_error_type nhóm lỗi phụ âm đầu, vần, dấu thanh, viết hoa, dấu câu hoặc bỏ sót/thêm. Điểm chính tả = $\max(0, 4 - \text{số lỗi} \times \text{penalty})$. Điểm hình thức và nội dung mặc định lần lượt 2.5/3 và 1.5/2 nếu không có override; sáng tạo là heuristic từ khóa/độ dài.

6.4 Gemini

OCR dùng model gemini-3.1-flash-lite và yêu cầu JSON original_text/fixed_text. Route chấm có xoay vòng nhiều GEMINI_API_KEYS khi quota 429 hoặc lỗi 503. Cần theo dõi chi phí, giới hạn kích thước ảnh/văn bản, rate limit và không log nội dung học sinh quá mức cần thiết.

7. Tiền xử lý ảnh và kiểm soát chất lượng

lib/image-processor.ts hiện thực toàn bộ pipeline server-side bằng Jimp và thao tác pixel thuần JavaScript.

Bước	Kỹ thuật	Mục đích
0	Đọc EXIF Orientation	Xoay ảnh điện thoại đúng chiều.
0.5	Projection profile + Otsu	Tìm và sửa góc nghiêng, có lọc grid line.
1	Resize	Giới hạn chiều rộng để cân bằng tốc độ/chất lượng.
2	White balance	Giảm sai lệch màu do ánh sáng.
3	Grayscale	Chuẩn hóa ảnh một kênh.
4	Shadow removal	Làm giảm bóng đổ nền giấy.
5	CLAHE	Tăng tương phản cục bộ.
6	Sharpen	Làm rõ nét chữ.
7	Quality assessment	Đo blur, sáng, resolution, tỷ lệ pixel tối/vùng chữ và cảnh báo.
8	Threshold	Nhị phân hóa trước khi xuất JPEG base64.

7.1 Điểm cần lưu ý

- Pipeline xử lý pixel trong Node.js có thể chiếm CPU/RAM khi nhiều ảnh lớn; cần giới hạn payload, queue hoặc worker process.
- Chất lượng OCR hiện phụ thuộc Gemini; chưa có local OCR engine độc lập.
- Client tự nén lại ảnh gốc để gửi OCR, không phải ảnh processedImage. Nếu mục tiêu là tăng OCR, cần xác nhận đây có phải chủ đích hay chuyển sang dùng ảnh đã xử lý.

8. MCP Xiaozhi và đọc chính tả

mcp_service/main.py chạy FastAPI, vừa public /mcp HTTP vừa tạo WebSocket client kết nối endpoint Xiaozhi. Nó hỗ trợ JSON-RPC protocol version 2024-11-05.

8.1 Công cụ MCP

Tên tool	Input	Hành vi
vihand.save_dictation_session	title, passage, className, teacherName, summary, logs	POST tới /api/dictation/sessions, lưu phiên completed và trả thông báo.
vihand.get_dictation_sessions	className, limit	GET /api/dictation/sessions, trả danh sách phiên gần nhất.

8.2 Luồng JSON-RPC

- initialize: trả serverInfo, capabilities tools và ROLE_INSTRUCTIONS.
- tools/list: trả định nghĩa hai tool.
- tools/call: định tuyến theo tên tool, gọi HTTP Next.js bằng httpx timeout 10 giây.
- ping/notifications: phản hồi tối thiểu để duy trì kết nối Xiaozhi.

8.3 Cấu hình

MCP_ENDPOINT và VIHAND_API_URL được đọc từ môi trường. Tập .env.example hiện có dữ liệu giống token thực tế; tài liệu/commit không được chứa endpoint có credential. Phải revoke credential, thay bằng MCP_ENDPOINT=wss://...?token=<REDACTED>, rồi cấu hình secret qua secret store.

9. Triển khai, cấu hình và vận hành

Dự án cung cấp Dockerfile cho web, start_all.bat cho phát triển Windows và nhiều script triển khai Raspberry Pi.

9.1 Biến môi trường

Biến	Dùng tại	Mục đích
DATABASE_URL	Prisma/Docker	Chuỗi SQLite, Docker mặc định file:/data/vihand.db.
GEMINI_API_KEYS	API OCR và grade	Danh sách khóa Gemini phân tách bằng dấu phẩy.
VIT5_SERVICE_URL	API grade, warmup	URL FastAPI ViT5, mặc định http://localhost:8000.
ADMIN_SECRET_KEY	admin-register	Mã tạo admin; bắt buộc đặt secret mạnh bên ngoài source.
MCP_ENDPOINT	MCP service	WebSocket endpoint Xiaozhi có token.
VIHAND_API_URL	MCP service	Base URL Next.js mà MCP gọi.

9.2 Docker web

Dockerfile dùng build nhiều stage trên node:20-slim, cài toolchain để build better-sqlite3, prisma generate, npm run build, rồi copy standalone output/public/static/schema/migrations/node_modules. Runtime là nextjs non-root, expose 7860. Image cần start.sh ở build context; cần kiểm tra tệp này được track và migration chạy an toàn ở startup.

9.3 Lệnh vận hành

Web: npm run dev | Build: npm run build | Lint: npm run lint
Windows đầy đủ: start_all.bat
Python: cd python_service; python main.py
MCP: cd mcp_service; python -u main.py

9.4 Khả năng quan sát

- Health endpoints tồn tại cho ViT5 và MCP; nên bổ sung /health cho Next.js gồm kết nối DB và dependency AI.
- Ghi metric: số request OCR/chấm, thời gian, timeout/retry, Gemini token/quota, tỉ lệ fallback, RAM model và lỗi DB.
- Không log base64 ảnh, mật khẩu, key/token hoặc toàn văn bài học sinh vào log production.

PHẦN 10 - Bảo mật, riêng tư và rủi ro

Các mục sau là phát hiện từ hiện trạng mã nguồn, sắp theo mức ưu tiên xử lý. Đây là rủi ro quan trọng vì hệ thống xử lý tài khoản và bài viết của học sinh.

Mức	Phát hiện	Tác động	Hướng xử lý
P0	Credential MCP xuất hiện trong .env.example.	Lộ quyền kết nối trợ lý/endpoint.	Revoke/rotate ngay; xóa khỏi Git history nếu đã public; dùng secret manager.
P0	API CRUD, OCR và grade không xác thực/phân quyền server-side.	Người ngoài có thể đọc/sửa/xóa dữ liệu hoặc tiêu hao quota AI.	Session/JWT httpOnly, middleware, RBAC và ownership check cho mọi route.
P0	Password plaintext và password mặc định schema.	Lộ tài khoản khi DB/log bị truy cập; credential stuffing.	Argon2id/bcrypt, reset password, hash migration, policy và rate limit login.
P0	admin-register có secret fallback hard-code; link ẩn không bảo vệ endpoint.	Tạo admin trái phép nếu biết/đoán secret.	Bỏ endpoint sau bootstrap hoặc bắt buộc env secret, one-time bootstrap, audit.
P1	PATCH truyền nguyên body vào Prisma update.	Mass assignment: role, active, password và field không mong muốn có thể bị sửa.	Zod DTO allowlist theo role, audit log và validate ID.
P1	CORS ViT5 allow_origins=* cùng allow_credentials=True.	Tăng bề mặt gọi service từ origin không tin cậy.	Giới hạn origin cụ thể, mạng private/service auth.
P1	Ảnh và dữ liệu trẻ em gửi Gemini.	Rủi ro quyền riêng tư/tuân thủ.	Consent, minimization, retention, DPA, phân vùng dữ liệu và chính sách xóa.
P2	Next build bỏ qua lỗi TypeScript.	Lỗi hợp đồng dữ liệu lọt production.	Bỏ ignoreBuildErrors, bật lint/type-check trong CI.

10.1 Kiểm soát tối thiểu trước khi public

- Rotate toàn bộ secret, quét repo/ lịch sử và cấu hình production secrets tách biệt.
- Thực thi RBAC: admin quản trị user/lớp, teacher chỉ xem/lưu bài lớp mình, student chỉ xem dữ liệu của mình.
- Thiết lập giới hạn request/body/upload, CSRF nếu cookie session, rate limit cho login/OCR/grade.
- Mã hóa backup, chính sách retention/xóa, nhật ký truy cập và thông báo/đồng thuận phụ huynh theo quy định áp dụng.

Kiểm thử và lộ trình cải tiến (Phần 11)

Kho 02_Kich_ban_Thuc_nghiem chứa dataset, benchmark Gemini/OCR/ViT5, test script xử lý ảnh và kết quả kiểm thử. Đây là nền tảng tốt nhưng cần đưa các kiểm tra trọng yếu vào pipeline tự động.

11.1 Chiến lược kiểm thử đề xuất

Lớp kiểm thử	Phạm vi
Unit	classify_error_type, chunking, remove repetition, scoring thresholds, validate DTO, RBAC policy.
Integration	Prisma migrations/seed; API auth, CRUD ownership, grade fallback, MCP save/get.
E2E	Đăng nhập theo role; upload ảnh; OCR/chấm/lưu/lich sử; camera fallback; admin workflow.
Regression AI	Tập ảnh chuẩn, expected text/error/score band; so sánh accuracy, latency, hallucination theo version model.
Security	Không truy cập chéo lớp, không mass assignment, upload bomb/large image, secret scan, rate limit.
Performance	Nhiều ảnh đồng thời, cold start ViT5, CPU/RAM Raspberry Pi, SQLite lock và quota Gemini.

11.2 Lộ trình kỹ thuật

- Giai đoạn 1 - An toàn: revoke secret, auth session/RBAC, hash password, schema validation, rate limits.
- Giai đoạn 2 - Tin cậy: bỏ TypeScript ignore, typed API client, chuẩn hóa schema quan hệ, backup/migration/observability.
- Giai đoạn 3 - Hiệu năng: tách hàng đợi xử lý ảnh/AI, object storage, caching có TTL, kiểm soát concurrency ViT5.
- Giai đoạn 4 - Chất lượng AI: bộ benchmark versioned, teacher correction feedback loop, đánh giá fairness theo vùng/phương ngữ.

11.3 Tiêu chí nghiệm thu khuyến nghị

- 100% route dữ liệu yêu cầu xác thực và kiểm tra quyền/ownership bằng test tự động.
- Không còn secret thực, password rõ hoặc default admin credential trong source/tracked config.
- Build/lint/type-check xanh; migration và backup restore được kiểm tra trong môi trường staging.
- Tỷ lệ OCR/chấm, độ trễ P95 và tỷ lệ fallback được dashboard hóa với ngưỡng cảnh báo.

PHẦN 12 - Phụ lục: bản đồ tệp quan trọng

Danh sách này dùng để định hướng bảo trì; không bao gồm toàn bộ asset, dataset, hình ảnh và component UI sinh sẵn.

Tệp / thư mục	Nội dung chính
package.json	Scripts dev/build/start/lint và dependency web.
next.config.mjs	Standalone build, image unoptimized, hiện ignore TypeScript build errors.
app/layout.tsx	Metadata, font, Analytics, service worker registration.
app/page.tsx	Đăng nhập phía client.
app/teacher/grade/page.tsx	Orchestrator UI xử lý ảnh/OCR/chấm/lưu.
app/api/grade/route.ts	Điều phối ViT5, retry và Gemini fallback.
app/api/ocr/route.ts	Gemini OCR và parse JSON.
app/api/preprocess/route.ts	Endpoint pipeline Jimp.
lib/image-processor.ts	Chất lượng ảnh và biến đổi pixel.
lib/prisma.ts; prisma/schema.prisma	Kết nối ORM và mô hình SQLite.
python_service/main.py	ViT5 quantized, post-processing, scoring API.
mcp_service/main.py	MCP JSON-RPC/WebSocket và bridge Xiaozhi.
Dockerfile; start_all.bat	Đóng gói web và startup development Windows.
02_Kich_ban_Thuc_nghiem/	Kịch bản benchmark, ảnh test, notebook và tài liệu kết quả.

Kết luận

ViHand Grade đã có nền tảng chức năng rõ ràng và pipeline AI kết hợp hợp lý: xử lý ảnh, Gemini OCR, ViT5 chấm cục bộ, SQLite/Prisma lưu lịch sử và MCP mở rộng đọc chính tả. Trước khi triển khai công khai, ưu tiên bắt buộc là quản lý secret, xác thực/ủy quyền phía server, bảo vệ dữ liệu học sinh và đưa kiểm thử/quan sát vào quy trình phát hành.